Reviewer Pack — Free Entropy Lower Bound for Disjointness Communication Matr...

Entry 44428529acab · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-29 08:50:38 UTC

Free Entropy Lower Bound for Disjointness Communication Matrices

Entry ID: `44428529acab` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-04-29 08:50:38 UTC

1. Conjecture

Field A (mathematical branch): Free Probability Theory **Field B** (complexity object): Communication Complexity

Statement:

> The free entropy of the DISJOINTNESS communication matrix M_n is Ω(n), implying that its randomized communication complexity is Ω(n).

Rationale (proposer's reasoning):

> Free entropy quantifies the complexity of noncommutative distributions. For the DISJOINTNESS matrix, its high free entropy reflects the inherent structure requiring Ω(n) communication, mirroring Forster's signed-rank technique but via free probability's noncommutative framework.

Taxonomy category: `ACC_LB_via_WILLIAMS` (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `8eb0767688e8197b`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE

Barrier	Verdict	Confidence	LLM_1	LLM_2
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=0, elapsed=0.0s

5.1 Generated Python source

```python
import random
import math
from itertools import combinations

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def free_entropy(M):
        n = len(M)
        trace = sum(M[i][i] for i in range(n))
        det = 1.0
        for i in range(n):
            for j in range(i + 1, n):
                det *= M[i][j]
        if det <= 0:
            return float('-inf')  # Handle non-positive determinant to avoid math domain error
        return trace - math.log(det)

    def disjointness_matrix(n):
        M = [[0] * n for _ in range(n)]
        for i, j in combinations(range(n), 2):
            M[i][j] = random.choice([-1, 1])
            M[j][i] = -M[i][j]
        return M

    def instances_tested(n):
        return n * (n - 1) // 2

    n_values = [5, 10, 15, 20, 30, 40]
    total_metric_value = 0
    num_seeds = len(n_values)

    for n in n_values:
        M = disjointness_matrix(n)
        fe = free_entropy(M)
        if fe == float('-inf'):
            return {
                "metric_name": "free_entropy",
                "metric_value": None,
                "instances_tested": instances_tested(n),
```

```
                "conjecture_holds": False,
                "counterexample": "non-positive determinant"
            }
        total_metric_value += fe

    mean_metric_value = total_metric_value / num_seeds
    support_fraction = 1.0

    return {
        "metric_name": "free_entropy",
        "metric_value": mean_metric_value,
        "instances_tested": instances_tested(n),
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) /
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fract
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample='non-positive determinant' first_failing_seed={fi
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
nterexample': 'non-positive determinant'}
TRIAL: {'metric_name': 'free_entropy', 'metric_value': None, 'instances_tested': 435, 'conjecture_ho
positive determinant'}
TRIAL: {'metric_name': 'free_entropy', 'metric_value': None, 'instances_tested': 10, 'conjecture_hol
positive determinant'}
TRIAL: {'metric_name': 'free_entropy', 'metric_value': None, 'instances_tested': 435, 'conjecture_ho
positive determinant'}
TRIAL: {'metric_name': 'free_entropy', 'metric_value': None, 'instances_tested': 105, 'conjecture_ho
positive determinant'}
TRIAL: {'metric_name': 'free_entropy', 'metric_value': None, 'instances_tested': 45, 'conjecture_hol
positive determinant'}
TRIAL: {'metric_name': 'free_entropy', 'metric_value': None, 'instances_tested': 105, 'conjecture_ho
positive determinant'}
```

```
TRIAL: {'metric_name': 'free_entropy', 'metric_value': None, 'instances_tested': 45, 'conjecture_hol
positive determinant'}
TRIAL: {'metric_name': 'free_entropy', 'metric_value': None, 'instances_tested': 10, 'conjecture_hol
positive determinant'}
TRIAL: {'metric_name': 'free_entropy', 'metric_value': None, 'instances_tested': 45, 'conjecture_hol
positive determinant'}
TRIAL: {'metric_name': 'free_entropy', 'metric_value': None, 'instances_tested': 10, 'conjecture_hol
positive determinant'}
TRIAL: {'metric_name': 'free_entropy', 'metric_value': None, 'instances_tested': 190, 'conjecture_ho
positive determinant'}
TRIAL: {'metric_name': 'free_entropy', 'metric_value': 0.0, 'instances_tested': 780, 'conjecture_hol
RESULT: FALSIFIED counterexample='non-positive determinant' first_failing_seed=11
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> The metric definition bug is critical: free entropy requires log(det(M)) which is undefined for non-positive determinants. The trials show 'non-positive determinant' counterexamples, yet metric_value is None/0, implying the test fails to compute free entropy properly. This invalidates the metric measurement entirely.

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The metric computation failed for non-positive determinants, invalidating free entropy measurements. The test's counterexamples suggest issues with metric definition. | next: Fix metric to handle non-positive determinants and revalidate computations

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	101421
2	propose	ollama_remote	qwen3:8b	0	0	79488
3	preregistration	ollama_remote	qwen3:8b	0	0	24135
4	novelty	ollama_remote	qwen3:8b	0	0	20777
5	novelty	ollama_remote	qwen3:8b	0	0	12842
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14799
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8073
8	critic	ollama_remote	qwen3:8b	0	0	30155
9	judge	ollama_remote	qwen3:8b	0	0	15948

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 307639 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/44428529acab.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/44428529acab.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/44428529acab.tar.gz` (if generated)